

# MPI

# Introduction

Ernesto Sanchez

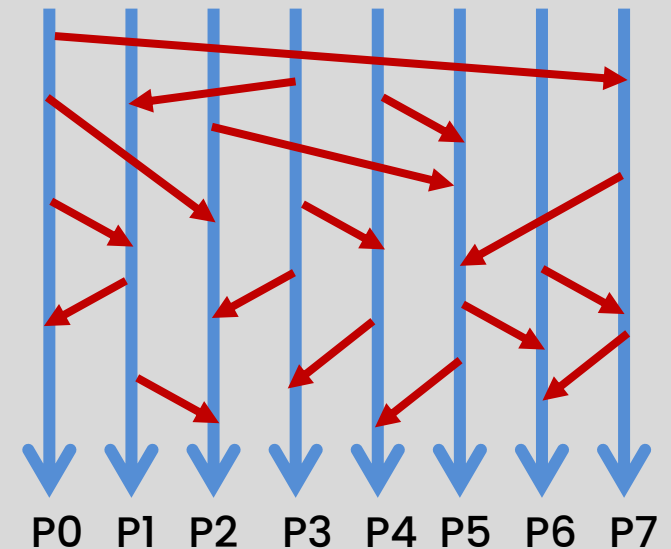
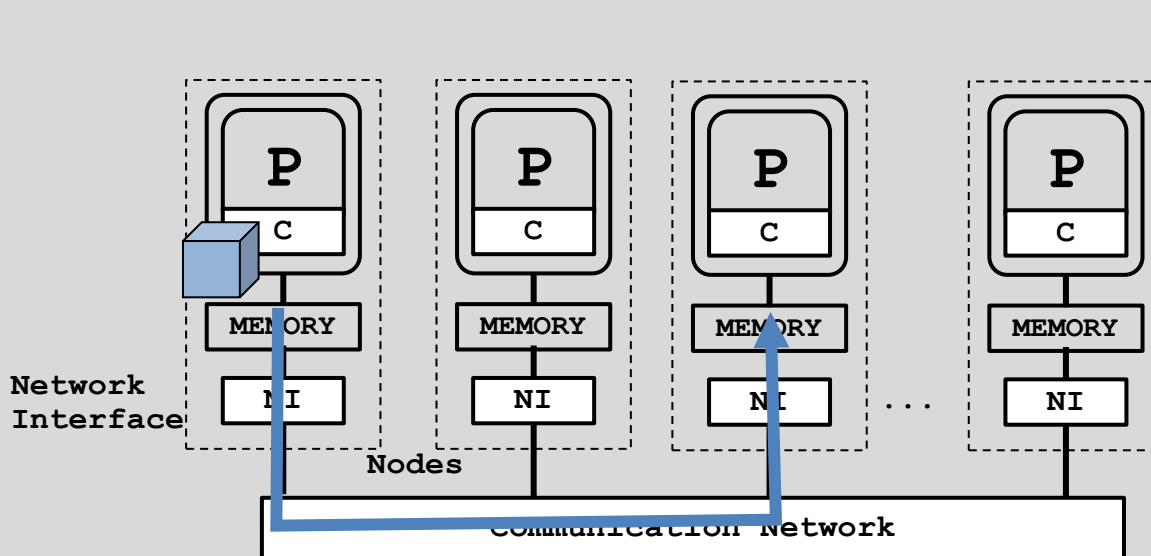
# Message Passing Interface (MPI) introduction

- Introduction to MPI
- MPI Communicator environment
- Communication types
  - Point to point
  - Collective
- Synchronization

# Introduction

In distribute memory computing systems, one processor cannot access to another processor memory directly.

The processors communicate via the NI (Network Interfaces) and then through the Communication network.



# Distribute-memory programming model

Communication between processors or processes is done only through messages (*message passing*).

Messages carry data between processes.

- The same program runs on all processes
- The program is written in a sequential way
- All variables are local to a process

For every message, it is necessary to define:

- Which process is sending the message?
- Where is the data on the sending message?
- Which are the data type and size of data to be sent?
- Which process is receiving the data?

# Message Passing Interface (MPI)

MPI is a message passing communication standard library.

- The library offers more than 500 functions to communicate between computing nodes
- About 20 functions are enough for performing parallel programming
- Includes efficient I/O parallelization

It supports:

- Point to point communication
- Collective communications

Different programming languages can be used, for example: C, Fortran, Python, R, etc.

Different implementations exist:

MVPICH, **Open MPI**, Intel MPI, etc.

- **Standard MPI**  
**version:5.0.7**  
- **HPC polito**  
**Open MPI 4.1.8**

<http://www.mpi-forum.org/docs/>



<https://www.open-mpi.org/>

# MPI standard

Open standard that targets the portability of parallel applications. The MPI forum is a nonprofit consortium of research, academic, and industry partners (more than 50 individuals and organizations).

It is necessary to use a standard programming library since:

- Clusters and supercomputers get improved very fast
- Applications using proprietary libraries are not portable
- Starting to create applications from scratch is very time consuming
- Staff mobility is very common among parallel programming experts

MPI is a dominant parallel programming model in the most of the today's clusters.

<https://www.mpi-forum.org/>

# MPI general features

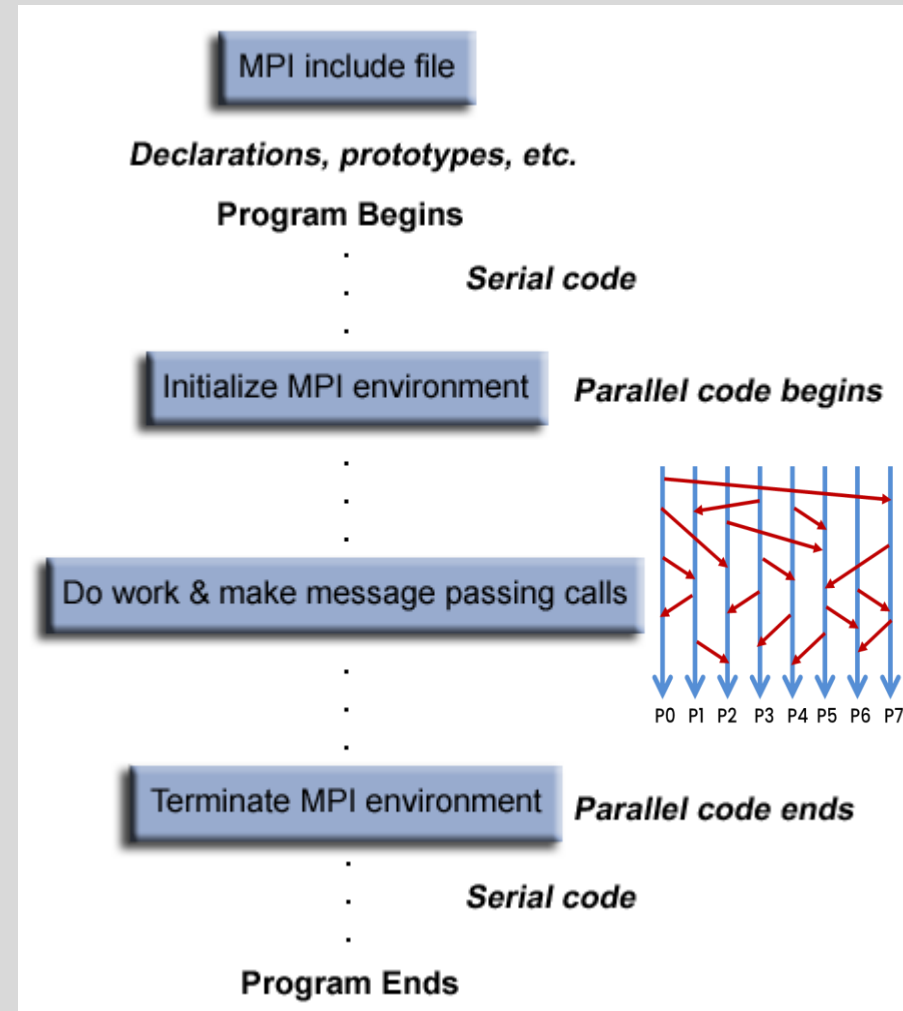
- Mainly designed for performance within large number of parallel computers, not including security features
- Supports various interconnection mechanisms between computing nodes
- The most benefits are related to message handling in a reliable way and adopting in-order arrivals, for example.
- MPI is not designed to handle any communication in computer networks, and it is not very good for very frequent establishing/closing connections
- Not good for internet chat clients, or Web service servers.

<https://www.mpi-forum.org/>

# Getting started

## *Start thinking in parallel*

- Write the program sequential
- Data exchange is done by sending and receiving data at specific execution time in the program
- No shared-memory
- After the initial setup:
  - Initialize the MPI environment
  - Run the parallel code
  - Conclude the parallelization
  - Return to run serial code.



<https://hpc-tutorials.llnl.gov/mpi/>

# Hola\_mundo\_MPI version

```
#include <stdio.h>
#include <mpi.h>

int main(int argc, char** argv)
{
    int rank, size;
    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    printf("hola mundo, i am %d out of %d \n",rank, size);
    MPI_Finalize();
    return 0;
}
```

Organize every program in a different folder containing:

Example: **/HPC2026/MPI/holaMPI**

- The C program (**mpi\_hola.c**)
- Submit-script (**submit-mpi\_hola.sh**)
- Gathered results (**output\_36386.txt**)

# Hola\_mundo\_MPI version

```
#include <stdio.h>
#include <mpi.h>

int main(int argc, char** argv)
{
    int rank, size;
    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    printf("hola mundo, i am %d out of %d \n",rank, size);
    MPI_Finalize();
    return 0;
}
```

Sequential C program classic components:

- Libraries
- Functions
- Variables.

The initial program can be compiled and debugged with normal C compilers.

# Hola\_mundo\_MPI version

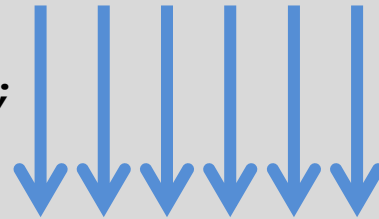
```
#include <stdio.h>
#include <mpi.h>

int main(int argc, char** argv)
{
    int rank, size;
    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    printf("hola mundo, i am %d out of %d \n", rank, size);
    MPI_Finalize();
    return 0;
}
```

- The MPI library needs to be included
- The parallel environment needs to be initialized
- Initialization and finalization functions of the environment initializes and terminates the parallel part of the program.

Arguments in the main() can be provided to the environment



# Hola\_mundo\_MPI version

```
#include <stdio.h>
#include <mpi.h>

int main(int argc, char** argv)
{
    int rank, size;
    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    printf("hola mundo, i am %d out of %d \n", rank, size);
    MPI_Finalize();
    return 0;
}
```

## **MPI\_Comm\_size()**

Returns the overall number of processes running in the parallel program. Value stores in the variable size

## **MPI\_COMM\_WORLD**

communicator structure that contains the information of all the processes created during the MPI\_Init function.

# Hola\_mundo\_MPI version

```
#include <stdio.h>
#include <mpi.h>

int main(int argc, char** argv)
{
    int rank, size;
    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    printf("hola mundo, i am %d out of %d \n", rank, size);
    MPI_Finalize();
    return 0;
}
```

## **MPI\_Comm\_size()**

Returns the overall number of processes running in the parallel program. Value stores in the variable *size*

## **MPI\_Comm\_rank()**

Returns for any process the unique identifier. It is stored in the variable *rank* with values (0 ... n-1)

## **MPI\_COMM\_WORLD**

communicator structure that contains the information of all the processes created during the MPI\_Init function.

# Compile and execution

Once the C program is written, compile this resorting to a MPI compiler, for example:

- mpicc -- Open MPI C wrapper compiler

```
...@hpc-legionlogin$ mpicc hola_MPI.c -o hola_MPI
```

- mpirun -- Execute serial and parallel jobs in Open MPI

```
...@ hpc-legionlogin$ mpirun ./hola_MPI
```

It is always better to run the program resorting to a submit script.

# SLURM script for *Hola\_mundo\_MPI*

## submit-mpi\_hola.sh

```
...@hpc-legionlogin$ sbatch  
submit-mpi_hola.sh
```

```
#!/bin/bash  
#SBATCH -J hola-MPI  
#SBATCH --time=0:00:02  
#SBATCH -n 4                ## number of tasks  
#SBATCH --mail-user=my_email@polito.it  
#SBATCH --mail-type=end  
#SBATCH --output=output_%j.txt  
## --- load SLURM modules ---  
module load openmpi/4.1.8_gcc11  
  
## --- Code compilation ---  
mpicc mpi_hola.c -o mpi_hola  
  
## --- Program execution ---  
mpirun -np $SLURM_NTASKS ./mpi_hola
```

# SLURM script Output

output-36386.out

...@ hpc-legionlogin\$ cat output-36386.txt

Loading openmpi/4.1.8\_gcc11

Loading requirement: gcc/11.5.0

Currently Loaded Modulefiles:

1) gcc/11.5.0 <aL> 2) openmpi/4.1.8\_gcc11

Key:

<module-tag> <aL>=auto-loaded

hola mundo, i am 1 out of 4 I am rank out of size

hola mundo, i am 3 out of 4

hola mundo, i am 2 out of 4

hola mundo, i am 0 out of 4

# MPI Communicator environment

When MPI is initialized (`MPI_Init()`), a communicator group is created.

A group is an ordered set of **N processes**, usually defined in the submit-script as:

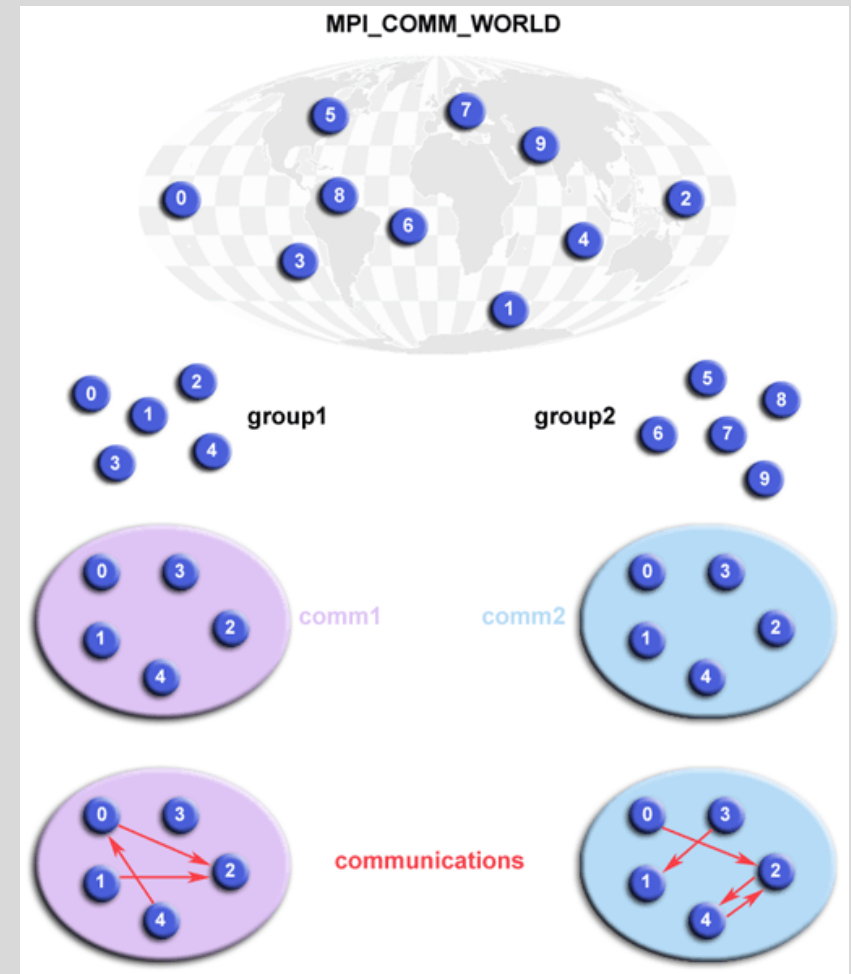
- `#SBATCH -n 4`
- `mpirun -n 4 MPI_hola`

Each process is associated with a unique integer **rank**.

Rank values start at zero and go to N-1, where N is the number of processes in the group.

**MPI\_COMM\_WORLD** is the handler for of all processes.

It is possible to create subgroups performing communications only for the specific processes inside.



[https://hpc-tutorials.llnl.gov/mpi/management\\_routines/](https://hpc-tutorials.llnl.gov/mpi/management_routines/)

# MPI main functions

## **MPI\_Init(&argc, &arg)**

Initializes the MPI execution environment. For C programs, MPI\_Init may be used to pass the command line arguments to all processes, although this is not required by the standard and is implementation dependent.

## **MPI\_Comm\_size(comm, &size)**

Returns the total number of MPI processes in the specified communicator (comm), such as MPI\_COMM\_WORLD. If the communicator is MPI\_COMM\_WORLD, then it represents the number of MPI tasks available to your application.

## **MPI\_Comm\_rank(comm, &rank)**

Returns the rank of the calling MPI process within the specified communicator. This rank is a unique integer value in the communicator.

## **MPI\_Finalize()**

Terminates the MPI execution environment. This function should be the last MPI routine called in every MPI program.

# MPI data types

MPI needs to know the data types of messages since it supports heterogeneous environments where it may be necessary to do on-the-fly data conversions.

For any message transfer to proceed, the data types on sender and receiver sides must match. Similar data types exist for the Fortran programming language.

| MPI datatype           | C equivalent           |
|------------------------|------------------------|
| MPI_SHORT              | short int              |
| MPI_INT                | int                    |
| MPI_LONG               | long int               |
| MPI_LONG_LONG          | long long int          |
| MPI_UNSIGNED_CHAR      | unsigned char          |
| MPI_UNSIGNED_SHORT     | unsigned short int     |
| MPI_UNSIGNED           | unsigned int           |
| MPI_UNSIGNED_LONG      | unsigned long int      |
| MPI_UNSIGNED_LONG_LONG | unsigned long long int |
| MPI_FLOAT              | float                  |
| MPI_DOUBLE             | double                 |
| MPI_LONG_DOUBLE        | long double            |
| MPI_BYTE               | char                   |

# MPI send

```
MPI_Send(  
    void* data,           // message data buffer  
    int count,            // # of elements  
    MPI_Datatype datatype, // MPI data type  
    int destination,      // destination rank  
    int tag,              // message tag  
    MPI_Comm communicator) // communicator
```

It is the basic function for sending messages.

It performs a blocking send until the message is received by the destination process. But, once the message is sent, the buffer can be safely modified by the sender.

# MPI receive

```
MPI_Recv(  
    void* data,                // message data buffer  
    int count,                 // max # of elements  
    MPI_Datatype datatype,     // MPI data type  
    int source,                // source rank  
    int tag,                   // message tag  
    MPI_Comm communicator,     // communicator  
    MPI_Status* status)       // status object
```

It performs a blocking receive for a message.

The source and tag arguments of `MPI_Recv()` may be equipped with the special constants ("wildcards"): `MPI_ANY_SOURCE` and `MPI_ANY_TAG`.

`MPI_Send()` and `MPI_Recv()` have blocking semantics, meaning that the buffer can be used safely after the function returns without altering any message in flight, and one can be sure that the message has been completely received after `MPI_Recv()`.

# MPI receive

```
MPI_Recv(  
    void* data,                // message data buffer  
    int count,                 // max # of elements  
    MPI_Datatype datatype,     // MPI data type  
    int source,                // source rank  
    int tag,                   // message tag  
    MPI_Comm communicator,     // communicator  
    MPI_Status* status)       // status object
```

```
typedef struct _MPI_Status {  
    int count;  
    int cancelled;  
    int MPI_SOURCE;  
    int MPI_TAG;  
    int MPI_ERROR;  
} MPI_Status, *PMPI_Status;
```

# Other MPI functions

## MPI Abort(comm,errorcode)

Terminates all MPI processes associated with the communicator.

## MPI Get\_processor\_name(&name,&resultlength)

Returns the processor name. Also returns the length of the name. The buffer for "name" must be at least MPI\_MAX\_PROCESSOR\_NAME characters in size.

## MPI Get\_version(&version,&subversion)

Returns the version and subversion of the MPI standard that's implemented by the library.

## MPI Initialized(&flag)

Indicates whether MPI\_Init has been called – returns flag as either logical true (1) or false(0).

## MPI Wtime()

Returns an elapsed wall clock time in seconds (double precision) on the calling processor.

## MPI Wtick()

Returns the resolution in seconds (double precision) of MPI\_Wtime.

## MPI Get\_count(MPI Status \*status, MPI Datatype datatype, int \*count)

Gets the number of received elements.

# *First communication program pingpong - 1*

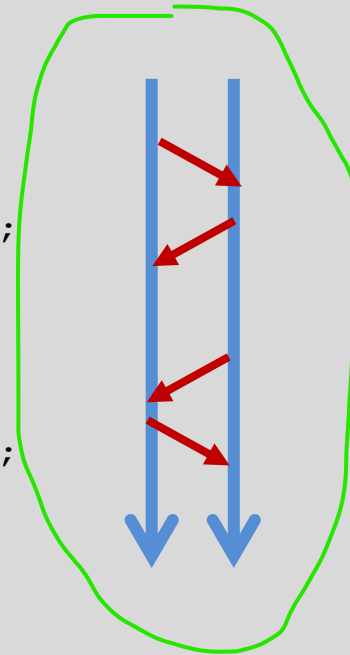
```
#include "mpi.h"
#include <stdio.h>
#include <stdlib.h>

int main (int argc, char *argv[])
{
    int numtasks, rank, dest, source, rc, count, tag=1;
    char inmsg, outmsg='x';
    MPI_Status Stat;

    MPI_Init(&argc,&argv);
    MPI_Comm_size(MPI_COMM_WORLD, &numtasks);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    ...
}
```

# First communication program pingpong - 2

```
if (rank == 0) {
    if (numtasks > 2)
        printf("Numtasks=%d. Only 2 needed. Ignoring extra...\n", numtasks);
    dest = 1;
    source = 1;
    rc = MPI_Send(&outmsg, 1, MPI_CHAR, dest, tag, MPI_COMM_WORLD);
    rc = MPI_Recv(&inmsg, 1, MPI_CHAR, source, tag, MPI_COMM_WORLD, &Stat);
}
else if (rank == 1) {
    dest = 0;
    source = 0;
    rc = MPI_Recv(&inmsg, 1, MPI_CHAR, source, tag, MPI_COMM_WORLD, &Stat);
    rc = MPI_Send(&outmsg, 1, MPI_CHAR, dest, tag, MPI_COMM_WORLD);
}
if (rank < 2) {
    rc = MPI_Get_count(&Stat, MPI_CHAR, &count);
    printf("Process %d: Received %d char(s) from process %d with tag %d \n", rank, count,
        Stat.MPI_SOURCE, Stat.MPI_TAG);
}
MPI_Finalize();
}
```



# MPI communication types

- Point to point
- Collective
  - Broadcast
  - Scatter
  - Gather
  - Reduce

# Point-to-point communication

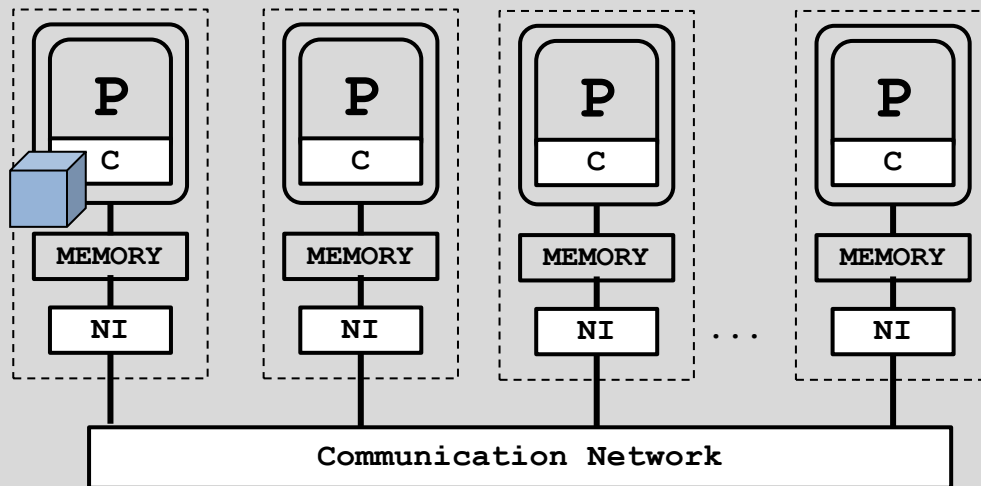
Communication is done through *messages*. Point-to-point communication takes place among exactly one sender and exactly one receiver processes. These two processes are identified uniquely by their ranks.

```
MPI_Send(  
    void* data,           // message data buffer  
    int count,            // # of elements  
    MPI_Datatype datatype, // MPI data type  
    int destination,      // destination rank  
    int tag,              // message tag  
    MPI_Comm communicator) // communicator
```

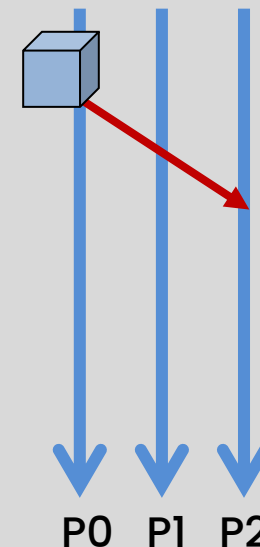
```
MPI_Recv(  
    void* data,           // message data buffer  
    int count,            // max # of elements  
    MPI_Datatype datatype, // MPI data type  
    int source,           // source rank  
    int tag,              // message tag  
    MPI_Comm communicator, // communicator  
    MPI_Status* status)   // status object
```

# Point-to-point communication

The message goes from a source process to a destination process.



```
MPI_Send(  
  My_data,  
  10, // # elems  
  MPI_INT,  
  2, //dest  
  1, //tag  
  MPI_COMM_WORLD)
```



```
MPI_Recv(  
  My_data,  
  10, //max elems  
  MPI_INT,  
  0, //source  
  1, //tag  
  MPI_COMM_WORLD,  
  Stat)
```

# Collective communication

Collective communications exchange data in different patterns among more than two processes within a communicator.

- . Broadcast
- . Scatter
- . Gather
- . Reduce

# Broadcast communication

One-to-many communication. The same data is sent to many or all the processors. All the process in the communicator are involved

```
int MPI_Bcast(  
    void* data,           // message data buffer  
    int count,            // # of elements in data buffer  
    MPI_Datatype datatype, // MPI data type  
    int root,              // source rank  
    MPI_Comm communicator) // communicator
```

MPI\_Bcast()

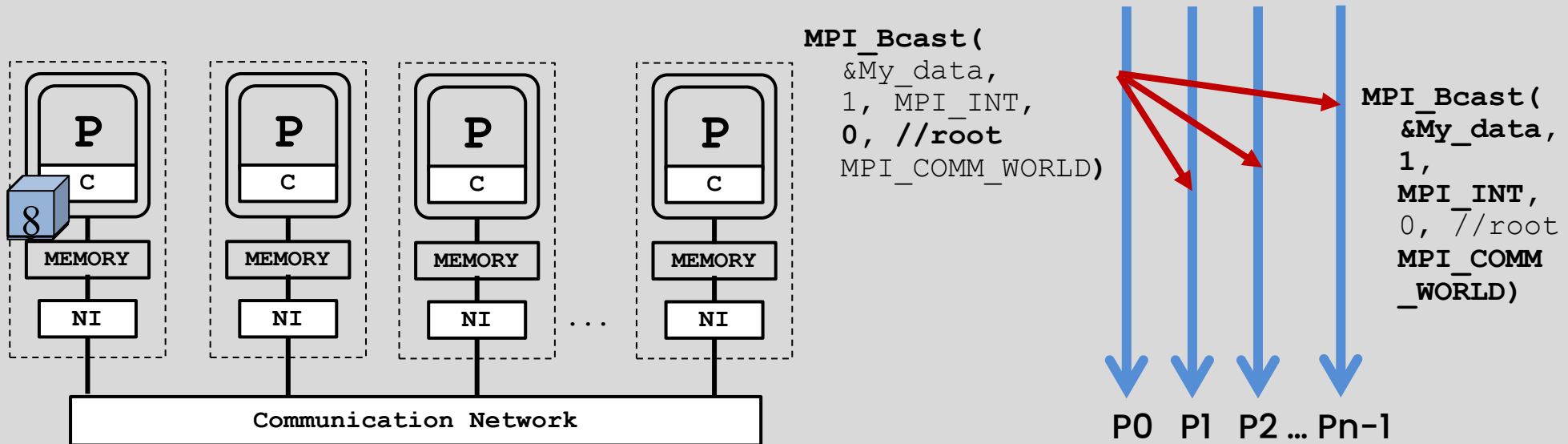
Is run by all the processes in the communicator. The root process sends data, while the others in the communicator receive this information.

# Broadcast communication

One-to-many communication. The same data is sent to many or all the processors. All the process in the communicator are involved

```
int MPI_Bcast(&buffer, count, datatype, root, comm)
```

Data movement operation. `MPI_Bcast()` sends a message from the process with rank “root” to all other processes in the group.



# Scatter communication

One-to-many.

Scatter distributes data of one process in separate parts to all the processes including itself within the scope of a communicator.

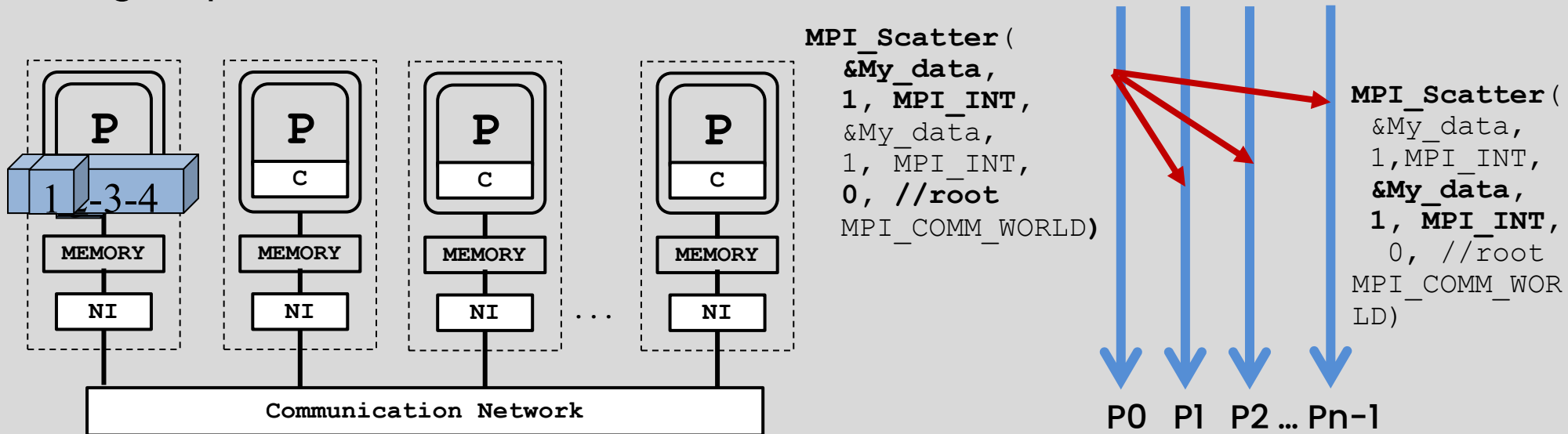
```
int MPI_Scatter (  
    void* sendbuf,          // send data buffer  
    int sendcnt,           // num. of elements sent to each process  
    MPI_Datatype datatype,  // MPI data type  
    void* recvbuf,         // receive data buffer  
    int recevcnt,          // num. elements in the receive buffer  
    MPI_Datatype datatype,  // MPI data type  
    int root,              // source rank  
    MPI_Comm communicator) // communicator
```

# Scatter communication

Scatter distributes data of one process in separate parts to all the processes including itself within the scope of a communicator.

```
int MPI_Scatter(&sendbuf, sendcnt, sendtype, &recvbuf,
recvcnt, recvtype, root, comm)
```

Distributes distinct messages from a single source task to each task in the group



# Gather communication

Many-To-One. Every process within the communicator sends its dataset to a specific process.

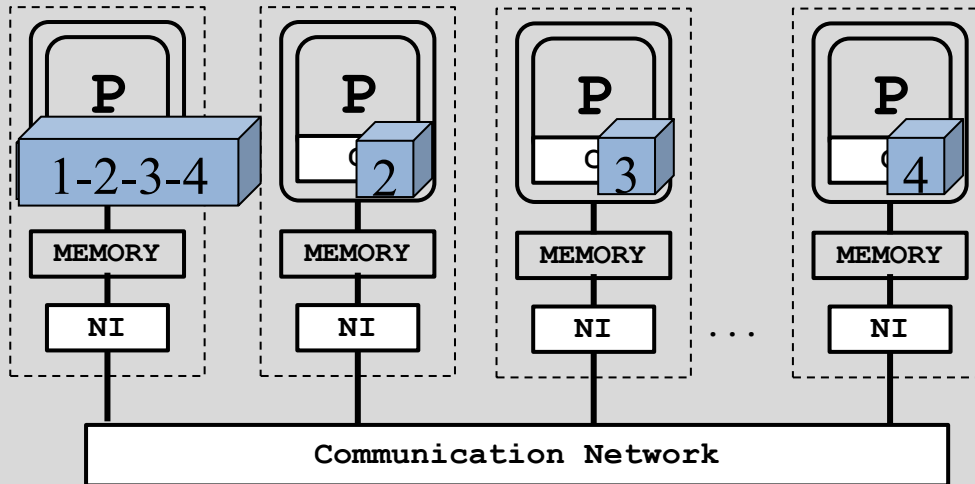
```
int MPI_Gather (  
    void* sendbuf,      // send data buffer  
    int sendcnt,        // num. of elements sent to each process  
    MPI_Datatype datatype, // MPI data type  
    void* recvbuf,      // receive data buffer  
    int recevcnt,        // num. of elements in the receive buffer  
    MPI_Datatype datatype, // MPI data type  
    int destination_rank, // destination rank  
    MPI_Comm communicator) // communicator
```

# Gather communication

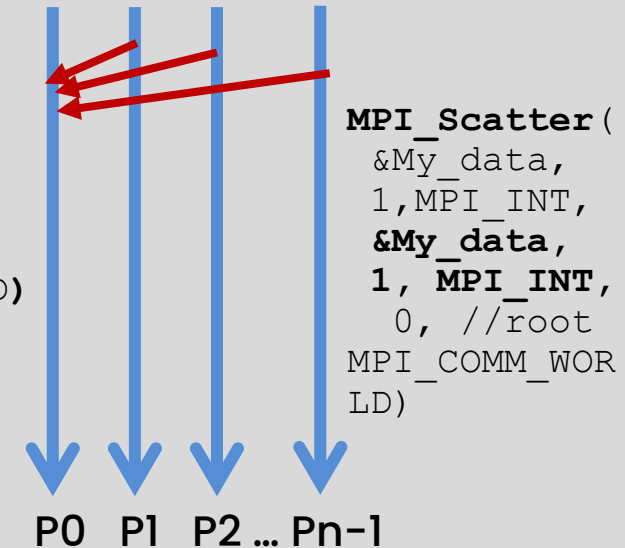
Many-To-One. Every process within the communicator sends its dataset to a specific process.

```
int MPI_Gather(&sendbuf, sendcnt, sendtype, &recvbuf,
recvcnt, recvtype, destination_rank, comm)
```

Distributes distinct messages from a single source task to each task in the group



```
MPI_Scatter(
    &My_data,
    1, MPI_INT,
    &My_data,
    1, MPI_INT,
    0, //root
    MPI_COMM_WORLD)
```



# Allgather communication

Many-To-many. Gathers data from all processes and distribute it to all the processes in the communicator.

```
int MPI_Allgather (  
    void* sendbuf,      // send data buffer  
    int sendcnt,        // num. of elements sent to each process  
    MPI_Datatype datatype, // MPI data type  
    void* recvbuf,      // receive data buffer  
    int recevcnt,       // num. of elements in the receive buffer  
    MPI_Datatype datatype, // MPI data type  
    MPI_Comm communicator) // communicator
```

# Reduction operations

Used to gather data, for example if different processes execute a different part of the same problem

Many-To-One. Reduce gathers information with a computation based on the available data from many or all other processes. Applies a reduction operation on all processes in the group and places the result in one process. The reduction operation may compute a SUM, find a MAX or MIN value, as well as a user defined operation.

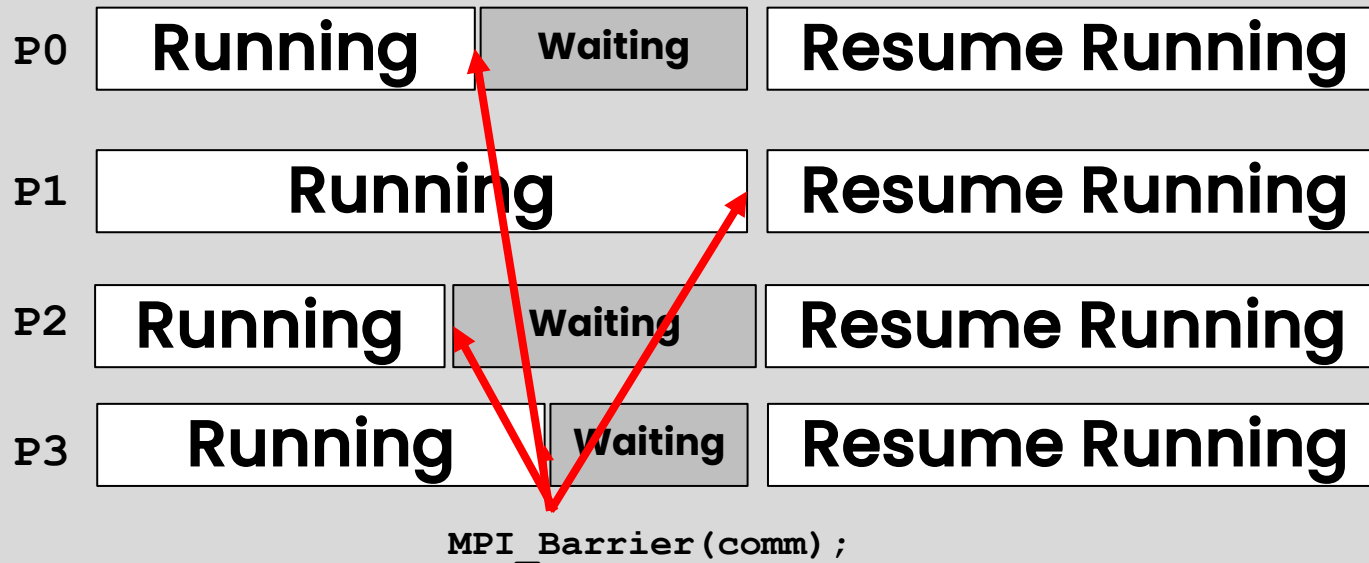
```
int MPI_Reduce (  
    void* sendbuf,           // send data buffer  
    void* recvbuf,          // receive data buffer  
    int count,               // number of elements sent to each process  
    MPI_Datatype datatype,   // MPI data type  
    MPI_Op op,               // reduce operation  
    int root,                // destination rank  
    MPI_Comm communicator)   // communicator
```

**MPI\_Op**: **MPI\_MAX**: maximum, **MPI\_MIN** minimum, **MPI\_SUM** sum, **MPI\_PROD** product in integer or float values. **MPI LAND** logical AND, etc.

# Barrier synchronization

`MPI_Barrier()` creates a point of barrier synchronization. It requires that all the processes reach that point in their own code.

```
MPI_Barrier(  
    MPI_Comm communicator)    // communicator
```



# Non blocking Point-to-point communication

Non blocking point-to-point communication returns immediately without confirming the message reception or completion.

```
MPI_Isend(  
    void* data,                // message data buffer  
    int count,                 // # of elements  
    MPI_Datatype datatype,     // MPI data type  
    int destination,           // destination rank  
    int tag,                   // message tag  
    MPI_Comm communicator,     // communicator  
    MPI_Request *request)      // communication request
```

```
MPI_Irecv(  
    void* data,                // message data buffer  
    int count,                 // max # of elements  
    MPI_Datatype datatype,     // MPI data type  
    int source,                // source rank  
    int tag,                   // message tag  
    MPI_Comm communicator,     // communicator  
    MPI_Status* status,        // status object  
    MPI_Request *recv_reqst)   // communication request
```

# Non blocking Point-to-point communication

Non blocking point-to-point communication returns immediately without confirming the message reception or completion.

```
MPI_Wait(  
    MPI_Request *request,    // communication request  
    MPI_Status* status)     // status object  
  
MPI_Test(  
    MPI_Request *request,    // communication request  
    int *flag,               // true if operation completed  
    MPI_Status* status)     // status object
```